

3.10 在机器级程序中将控制与数据结合起来

到目前为止，我们已经分别讨论机器级代码如何实现程序的控制部分和如何实现不同的数据结构。在本节中，我们会看看数据和控制如何交互。首先，深入审视一下指针，它是 C 编程语言中最重要的概念之一，但是许多程序员对它的理解都非常浅显。我们复习符号调试器 GDB 的使用，用它仔细检查机器级程序的详细运行。接下来，看看理解机器级程序如何帮助我们研究缓冲区溢出，这是现实世界许多系统中一种很重要的安全漏洞。最后，查看机器级程序如何实现函数要求的栈空间大小在每次执行时都可能不同的情况。

3.10.1 理解指针

指针是 C 语言的一个核心特色。它们以一种统一方式，对不同数据结构中的元素产生引用。对于编程新手来说，指针总是会带来很多的困惑，但是基本概念其实非常简单。在此，我们重点介绍一些指针和它们映射到机器代码的关键原则。

- 每个指针都对应一个类型。这个类型表明该指针指向的是哪一类对象。以下面的指针声明为例：

```
int *ip;
char **cpp;
```

变量 `ip` 是一个指向 `int` 类型对象的指针，而 `cpp` 指针指向的对象自身就是一个指向 `char` 类型对象的指针。通常，如果对象类型为 `T`，那么指针的类型为 `T*`。特殊的 `void *` 类型代表通用指针。比如说，`malloc` 函数返回一个通用指针，然后通过显式强制类型转换或者赋值操作那样的隐式强制类型转换，将它转换成一个有类型的指针。指针类型不是机器代码中的一部分；它们是 C 语言提供的一种抽象，帮助程序员避免寻址错误。

- 每个指针都有一个值。这个值是某个指定类型的对象的地址。特殊的 `NULL(0)` 值表示该指针没有指向任何地方。
- 指针用 `&` 运算符创建。这个运算符可以应用到任何 `lvalue` 类的 C 表达式上，`lvalue` 意指可以出现在赋值语句左边的表达式。这样的例子包括变量以及结构、联合和数组的元素。我们已经看到，因为 `leaq` 指令是设计用来计算内存引用的地址的，`&` 运算符的机器代码实现常常用这条指令来计算表达式的值。
- `*` 操作符用于间接引用指针。其结果是一个值，它的类型与该指针的类型一致。间接引用是用内存引用来实现的，要么是存储到一个指定的地址，要么是从指定的地址读取。
- 数组与指针紧密联系。一个数组的名字可以像一个指针变量一样引用（但是不能修改）。数组引用（例如 `a[3]`）与指针运算和间接引用（例如 `*(a+3)`）有一样的效果。数组引用和指针运算都需要用对象大小对偏移量进行伸缩。当我们写表达式 `p+i`，这里指针 `p` 的值为 `p`，得到的地址计算为 `p+L·i`，这里 `L` 是与 `p` 相关联的数据类型的大小。
- 将指针从一种类型强制转换成另一种类型，只改变它的类型，而不改变它的值。强制类型转换的一个效果是改变指针运算的伸缩。例如，如果 `p` 是一个 `char *` 类型的指针，它的值为 `p`，那么表达式 `(int *)p+7` 计算为 `p+28`，而 `(int *) (p+7)` 计算为 `p+7`。（回想一下，强制类型转换的优先级高于加法。）

- 指针也可以指向函数。这提供了一个很强大的存储和向代码传递引用的功能，这些引用可以被程序的某个其他部分调用。例如，如果我们有一个函数，用下面这个原型定义：

```
int fun(int x, int *p);
```

然后，我们可以声明一个指针 `fp`，将它赋值为这个函数，代码如下：

```
int (*fp)(int, int *);
fp = fun;
```

然后用这个指针来调用这个函数：

```
int y = 1;
int result = fp(3, &y);
```

函数指针的值是该函数机器代码表示中第一条指令的地址。

给 C 语言初学者 函数指针

函数指针声明的语法对程序员新手来说特别难以理解。对于以下声明：

```
int (*f)(int*);
```

要从里(从“`f`”开始)往外读。因此，我们看到像“`(*f)`”表明的那样，`f` 是一个指针；而“`(*f)(int*)`”表明 `f` 是一个指向函数的指针，这个函数以一个 `int*` 作为参数。最后，我们看到，它是指向以 `int*` 为参数并返回 `int` 的函数的指针。

`*f` 两边的括号是必需的，否则声明变成

```
int *f(int*);
```

它会被解读成

```
(int *) f(int*);
```

也就是说，它会被解释成一个函数原型，声明了一个函数 `f`，它以一个 `int*` 作为参数并返回一个 `int*`。

Kernighan 和 Ritchie [61, 5.12 节] 提供了一个有关阅读 C 声明的很有帮助的教程。

3.10.2 应用：使用 GDB 调试器

GNU 的调试器 GDB 提供了许多有用的特性，支持机器级程序的运行时评估和分析。对于本书中的示例和练习，我们试图通过阅读代码，来推断出程序的行为。有了 GDB，可以观察正在运行的程序，同时又对程序的执行有相当的控制，这使得研究程序的行为变为可能。

图 3-39 给出了一些 GDB 命令的例子，帮助研究机器级 x86-64 程序。先运行 `OBJDUMP` 来获得程序的反汇编版本，是很有好处的。我们的示例都基于对文件 `prog` 运行 GDB，程序的描述和反汇编见 3.2.3 节。我们用下面的命令行来启动 GDB：

```
linux> gdb prog
```

通常的方法是在程序中感兴趣的地方附近设置断点。断点可以设置在函数入口后面，或是一个程序的地址处。程序在执行过程中遇到一个断点时，程序会停下来，并将控制返回给用户。在断点处，我们能够以各种方式查看各个寄存器和内存位置。我们也可以单步跟踪程序，一次只执行几条指令，或是前进到下一个断点。

命令	效果
开始和停止 quit run kill	退出 GDB 运行程序(在此给出命令行参数) 停止程序
断点 break multstore break * 0x400540 delete 1 delete	在函数 multstore 入口处设置断点 在地址 0x400540 处设置断点 删除断点 1 删除所有断点
执行 stepi stepi 4 nexti continue finish	执行 1 条指令 执行 4 条指令 类似于 stepi, 但以函数调用为单位 继续执行 运行到当前函数返回
检查代码 disas disas multstore disas 0x400544 disas 0x400540,0x40054d print /x \$rip	反汇编当前函数 反汇编函数 multstore 反汇编位于地址 0x400544 附近的函数 反汇编指定地址范围内的代码 以十六进制输出程序计数器的值
检查数据 print \$rax print /x \$rax print /t \$rax print 0x100 print /x 555 print /x (\$rsp+ 8) print *(long *) 0x7fffffff818 print *(long *) (\$rsp+ 8) x/2g 0x7fffffff818 x/20bmultstore	以十进制输出%rax 的内容 以十六进制输出%rax 的内容 以二进制输出%rax 的内容 输出 0x100 的十进制表示 输出 555 的十六进制表示 以十六进制输出%rsp 的内容加上 8 输出位于地址 0x7fffffff818 的长整数 输出位于地址%rsp+8 处的长整数 检查从地址 0x7fffffff818 开始的双(8 字节)字 检查函数 multstore 的前 20 个字节
有用的信息 info frame info registers help	有关当前栈帧的信息 所有寄存器的值 获取有关 GDB 的信息

图 3-39 GDB 命令示例。说明了一些 GDB 支持机器级程序调试的方式

正如我们的示例表明的那样，GDB 的命令语法有点晦涩，但是在线帮助信息(用 GDB 的 help 命令调用)能克服这些毛病。相对于使用命令行接口来访问 GDB，许多程序员更愿意使用 DDD，它是 GDB 的一个扩展，提供了图形用户界面。

3. 10. 3 内存越界引用和缓冲区溢出

我们已经看到，C 对于数组引用不进行任何边界检查，而且局部变量和状态信息(例如保存的寄存器值和返回地址)都存放在栈中。这两种情况结合到一起就能导致严重的程序错误，对越界的数组元素的写操作会破坏存储在栈中的状态信息。当程序使用这个被破

坏的状态，试图重新加载寄存器或执行 `ret` 指令时，就会出现很严重的错误。

一种特别常见的状态破坏称为缓冲区溢出(buffer overflow)。通常，在栈中分配某个字符数组来保存一个字符串，但是字符串的长度超出了为数组分配的空间。下面这个程序示例就说明了这个问题：

```
/* Implementation of library function gets() */
char *gets(char *s)
{
    int c;
    char *dest = s;
    while ((c = getchar()) != '\n' && c != EOF)
        *dest++ = c;
    if (c == EOF && dest == s)
        /* No characters read */
        return NULL;
    *dest++ = '\0'; /* Terminate string */
    return s;
}

/* Read input line and write it back */
void echo()
{
    char buf[8]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

前面的代码给出了库函数 `gets` 的一个实现，用来说明这个函数的严重问题。它从标准输入读入一行，在遇到一个回车换行字符或某个错误情况时停止。它将这个字符串复制到参数 `s` 指明的位置，并在字符串结尾加上 `null` 字符。在函数 `echo` 中，我们使用了 `gets`，这个函数只是简单地从标准输入中读入一行，再把它回送到标准输出。

`gets` 的问题是它没有办法确定是否为保存整个字符串分配了足够的空间。在 `echo` 示例中，我们故意将缓冲区设得非常小——只有 8 个字节长。任何长度超过 7 个字符的字符串都会导致写越界。

检查 GCC 为 `echo` 产生的汇编代码，看看栈是如何组织的：

```
void echo()
1  echo:
2      subq    $24, %rsp           Allocate 24 bytes on stack
3      movq    %rsp, %rdi          Compute buf as %rsp
4      call    gets               Call gets
5      movq    %rsp, %rdi          Compute buf as %rsp
6      call    puts               Call puts
7      addq    $24, %rsp           Deallocate stack space
8      ret                        Return
```

图 3-40 画出了 `echo` 执行时栈的组织。该程序把栈指针减去了 24(第 2 行)，在栈上分配了 24 个字节。字符数组 `buf` 位于栈顶，可以看到，`%rsp` 被复制到 `%rdi` 作为调用 `gets` 和 `puts` 的参数。这个调用的参数和存储的返回指针之间的 16 字节是未被使用的。只要用户输入不超过 7 个字符，`gets` 返回的字符串(包括结尾的 `null`)就能够放进为 `buf` 分配的

空间里。不过，长一些的字符串就会导致 `gets` 覆盖栈上存储的某些信息。随着字符串变长，下面的信息会被破坏：

输入的字符数量	附加的被破坏的状态
0~7	无
9~23	未被使用的栈空间
24~31	返回地址
32+	caller 中保存的状态

字符串到 23 个字符之前都没有严重的后果，但是超过以后，返回指针的值以及更多可能的保存状态会被破坏。如果存储的返回地址的值被破坏了，那么 `ret` 指令(第 8 行)会导致程序跳转到一个完全意想不到的位置。如果只看 C 代码，根本就不可能看出会有上面这些行为。只有通过研究机器代码级别的程序才能理解像 `gets` 这样的函数进行的内存越界写的影响。

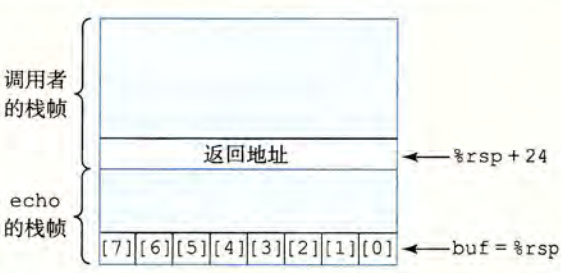


图 3-40 `echo` 函数的栈组织。字符数组 `buf` 就在保存的状态下面。对 `buf` 的越界写会破坏程序的状态

我们的 `echo` 代码很简单，但是有点太随意了。更好一点的版本是使用 `fgets` 函数，它包括一个参数，限制待读入的最大字节数。家庭作业 3.71 要求你写出一个能处理任意长度输入字符串的 `echo` 函数。通常，使用 `gets` 或其他任何能导致存储溢出的函数，都是不好的编程习惯。不幸的是，很多常用的库函数，包括 `strcpy`、`strcat` 和 `sprintf`，都有一个属性——不需要告诉它们目标缓冲区的大小，就产生一个字节序列[97]。这样的情况就会导致缓冲区溢出漏洞。

 **练习题 3.46** 图 3-41 是一个函数的(不太好的)实现，这个函数从标准输入读入一行，将字符串复制到新分配的存储中，并返回一个指向结果的指针。

考虑下面这样的场景。调用过程 `get_line`，返回地址等于 `0x400076`，寄存器 `%rbx` 等于 `0x0123456789ABCDEF`。输入的字符串为“0123456789012345678901234”。程序会因为段错误(segmentation fault)而中止。运行 GDB，确定错误是在执行 `get_line` 的 `ret` 指令时发生的。

- A. 填写下图，尽可能多地说明在执行完反汇编代码中第 3 行指令后栈的相关信息。在右边标注出存储在栈中的数字含意(例如“返回地址”)，在方框中写出它们的十六进制值(如果知道的话)。每个方框都代表 8 个字节。指出 `%rsp` 的位置。记住，字符 0~9 的 ASCII 代码是 `0x3~0x39`。

00 00 00 00 00 40 00 76	返回地址

- B. 修改你的图，展现调用 `gets` 的影响(第 5 行)。
C. 程序应该试图返回到什么地址？

- D. 当 `get_line` 返回时, 哪个(些)寄存器的值被破坏了?
- E. 除了可能会缓冲区溢出以外, `get_line` 的代码还有哪两个错误?

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

a) C代码

```
char *get_line()
1  0000000000400720 <get_line>:
2      400720: 53                                push    %rbx
3      400721: 48 83 ec 10                       sub     $0x10,%rsp
      Diagram stack at this point
4      400725: 48 89 e7                          mov     %rsp,%rdi
5      400728: e8 73 ff ff                       callq   4006a0 <gets>
      Modify diagram to show stack contents at this point
```

b) 对 `gets` 调用的反汇编

图 3-41 练习题 3.46 的 C 和反汇编代码

缓冲区溢出的一个更加致命的使用就是让程序执行它本来不愿意执行的函数。这是一种最常见的通过计算机网络攻击系统安全的方法。通常, 输入给程序一个字符串, 这个字符串包含一些可执行代码的字节编码, 称为攻击代码(exploit code), 另外, 还有一些字节会用一个指向攻击代码的指针覆盖返回地址。那么, 执行 `ret` 指令的效果就是跳转到攻击代码。

在一种攻击形式中, 攻击代码会使用系统调用启动一个 `shell` 程序, 给攻击者提供一组操作系统函数。在另一种攻击形式中, 攻击代码会执行一些未授权的任务, 修复对栈的破坏, 然后第二次执行 `ret` 指令, (表面上)正常返回到调用者。

让我们来看一个例子, 在 1988 年 11 月, 著名的 Internet 蠕虫病毒通过 Internet 以四种不同的方法获取对许多计算机的访问。一种是对 `finger` 守护进程 `fingerd` 的缓冲区溢出攻击, `fingerd` 服务 `FINGER` 命令请求。通过以一个适当的字符串调用 `FINGER`, 蠕虫可以使远程的守护进程缓冲区溢出并执行一段代码, 让蠕虫访问远程系统。一旦蠕虫获得了对系统的访问, 它就能自我复制, 几乎完全地消耗掉机器上所有的计算资源。结果, 在安全专家制定出如何消除这种蠕虫的方法之前, 成百上千的机器实际上都瘫痪了。这种蠕虫的始作俑者最后被抓住并被起诉。时至今日, 人们还是不断地发现遭受缓冲区溢出攻击的系统安全漏洞, 这更加突显了仔细编写程序的必要性。任何到外部环境的接口都应该是“防弹的”, 这样, 外部代理的行为才不会导致系统出现错误。

旁注 蠕虫和病毒

蠕虫和病毒都试图在计算机中传播它们自己的代码段。正如 Spafford[105]所述,蠕虫(worm)可以自己运行,并且能够将自己的等效副本传播到其他机器。病毒(virus)能将自己添加到包括操作系统在内的其他程序中,但它不能独立运行。在一些大众媒体中,“病毒”用来指各种在系统间传播攻击代码的策略,所以你可能会听到人们把本来应该叫做“蠕虫”的东西称为“病毒”。

3.10.4 对抗缓冲区溢出攻击

缓冲区溢出攻击的普遍发生给计算机系统造成了许多的麻烦。现代的编译器和操作系统实现了很多机制,以避免遭受这样的攻击,限制入侵者通过缓冲区溢出攻击获得系统控制的方式。在本节中,我们会介绍一些 Linux 上最新 GCC 版本所提供的机制。

1. 栈随机化

为了在系统中插入攻击代码,攻击者既要插入代码,也要插入指向这段代码的指针,这个指针也是攻击字符串的一部分。产生这个指针需要知道这个字符串放置的栈地址。在过去,程序的栈地址非常容易预测。对于所有运行同样程序和操作系统版本的系统来说,在不同的机器之间,栈的位置是相当固定的。因此,如果攻击者可以确定一个常见的 Web 服务器所使用的栈空间,就可以设计一个在许多机器上都能实施的攻击。以传染病来打个比方,许多系统都容易受到同一种病毒的攻击,这种现象常被称作安全单一化(security monoculture)[96]。

栈随机化的思想使得栈的位置在程序每次运行时都有变化。因此,即使许多机器都运行同样的代码,它们的栈地址都是不同的。实现的方式是:程序开始时,在栈上分配一段 $0 \sim n$ 字节之间的随机大小的空间,例如,使用分配函数 `alloca` 在栈上分配指定字节数量的空间。程序不使用这段空间,但是它会导致程序每次执行时后续的栈位置发生了变化。分配的范围 n 必须足够大,才能获得足够多的栈地址变化,但是又要足够小,不至于浪费程序太多的空间。

下面的代码是一种确定“典型的”栈地址的方法:

```
int main() {
    long local;
    printf("local at %p\n", &local);
    return 0;
}
```

这段代码只是简单地打印出 `main` 函数中局部变量的地址。在 32 位 Linux 上运行这段代码 10 000 次,这个地址的变化范围为 `0xff7fc59c` 到 `0xfffffd09c`,范围大小大约是 2^{23} 。在更新一点儿的机器上运行 64 位 Linux,这个地址的变化范围为 `0x7fff0001b698` 到 `0x7fffff001b698`,范围大小大约是 2^{32} 。

在 Linux 系统中,栈随机化已经变成了标准行为。它是更大的一类技术中的一种,这类技术称为地址空间布局随机化(Address-Space Layout Randomization),或者简称 ASLR [99]。采用 ASLR,每次运行时程序的不同部分,包括程序代码、库代码、栈、全局变量和堆数据,都会被加载到内存的不同区域。这就意味着在一台机器上运行一个程序,与在其他机器上运行同样的程序,它们的地址映射大相径庭。这样才能够对抗一些形式的攻击。

然而，一个执著的攻击者总是能够用蛮力克服随机化，他可以反复地用不同的地址进行攻击。一种常见的把戏就是在实际的攻击代码前插入很长一段的 `nop` (读作“no op”，no operation 的缩写) 指令。执行这种指令除了对程序计数器加一，使之指向下一条指令之外，没有任何的效果。只要攻击者能够猜中这段序列中的某个地址，程序就会经过这个序列，到达攻击代码。这个序列常用的术语是“空操作雪橇(nop sled)” [97]，意思是程序会“滑过”这个序列。如果我们建立一个 256 个字节的 `nop sled`，那么枚举 $2^{15} = 32\,768$ 个起始地址，就能破解 $n = 2^{23}$ 的随机化，这对于一个顽固的攻击者来说，是完全可行的。对于 64 位的情况，要尝试枚举 $2^{24} = 16\,777\,216$ 就有点儿令人畏惧了。我们可以看到栈随机化和其他一些 ASLR 技术能够增加成功攻击一个系统的难度，因而大大降低了病毒或者蠕虫的传播速度，但是也不能提供完全的安全保障。

 **练习题 3.47** 在运行 Linux 版本 2.6.16 的机器上运行栈检查代码 10 000 次，我们获得地址的范围从最小的 `0xfffffb754` 到最大的 `0xfffffd754`。

- A. 地址的大概范围是多大？
- B. 如果我们尝试一个有 128 字节 `nop sled` 的缓冲区溢出，要想穷尽所有的起始地址，需要尝试多少次？

2. 栈破坏检测

计算机的第二道防线是能够检测到何时栈已经被破坏。我们在 `echo` 函数示例(图 3-40)中看到，破坏通常发生在当超越局部缓冲区的边界时。在 C 语言中，没有可靠的方法来防止对数组的越界写。但是，我们能够在发生了越界写的时候，在造成任何有害结果之前，尝试检测到它。

最近的 GCC 版本在产生的代码中加入了一种栈保护者(stack protector)机制，来检测缓冲区越界。其思想是在栈帧中任何局部缓冲区与栈状态之间存储一个特殊的金丝雀(canary)值^①，如图 3-42 所示 [26, 97]。这个金丝雀值，也称为哨兵值(guard value)，是在程序每次运行时随机产生的，因此，攻击者没有简单的办法能够知道它是什么。在恢复寄存器状态和从函数返回之前，程序检查这个金丝雀值是否被该函数的某个操作或者该函数调用的某个函数的某个操作改变了。如果是的，那么程序异常中止。



图 3-42 `echo` 函数具有栈保护者的栈组织(在数组 `buf` 和保存的状态之间放了一个特殊的“金丝雀”值。代码检查这个金丝雀值，确定栈状态是否被破坏)

最近的 GCC 版本会试着确定一个函数是否容易遭受栈溢出攻击，并且自动插入这种溢出检测。实际上，对于前面的栈溢出展示，我们不得不用命令行选项“`-fno-stack-protector`”来阻止 GCC 产生这种代码。当不用这个选项来编译 `echo` 函数时，也就是允许使用栈保护者，得到下面的汇编代码：

```
void echo()
1  echo:
2  subq    $24, %rsp           Allocate 24 bytes on stack
```

① 术语“金丝雀”源于历史上用这种鸟在煤矿中察觉有毒的气体。


```

3      movq    %fs:40, %rax           Retrieve canary
4      movq    %rax, 8(%rsp)         Store on stack
5      xorl    %eax, %eax            Zero out register
6      movq    %rsp, %rdi            Compute buf as %rsp
7      call    gets                  Call gets
8      movq    %rsp, %rdi            Compute buf as %rsp
9      call    puts                  Call puts
10     movq    8(%rsp), %rax          Retrieve canary
11     xorq    %fs:40, %rax          Compare to stored value
12     je      .L9                    If =, goto ok
13     call    __stack_chk_fail      Stack corrupted!
14     .L9:                               ok:
15     addq    $24, %rsp              Deallocate stack space
16     ret

```

这个版本的函数从内存中读出一个值(第3行),再把它存放在栈中相对于`%rsp`偏移量为8的地方。指令参数`%fs:40`指明金丝雀值是用段寻址(segmented addressing)从内存中读入的,段寻址机制可以追溯到80286的寻址,而在现代系统上运行的程序中已经很少见到了。将金丝雀值存放在一个特殊的段中,标志为“只读”,这样攻击者就不能覆盖存储的金丝雀值。在恢复寄存器状态和返回前,函数将存储在栈位置处的值与金丝雀值做比较(通过第11行的`xorq`指令)。如果两个数相同,`xorq`指令就会得到0,函数会按照正常的方式完成。非零的值表明栈上的金丝雀值被修改过,那么代码就会调用一个错误处理例程。

栈保护很好地防止了缓冲区溢出攻击破坏存储在程序栈上的状态。它只会带来很小的性能损失,特别是因为GCC只在函数中有局部`char`类型缓冲区的时候才插入这样的代码。当然,也有其他一些方法会破坏一个正在执行的程序的状态,但是降低栈的易受攻击性能够对抗许多常见的攻击策略。



练习题 3.48 函数`intlen`、`len`和`iptoa`提供了一种很纠结的方式,来计算表示一个整数所需要的十进制数字的个数。我们利用它来研究GCC栈保护者措施的一些情况。

```

int len(char *s) {
    return strlen(s);
}

void iptoa(char *s, long *p) {
    long val = *p;
    sprintf(s, "%ld", val);
}

int intlen(long x) {
    long v;
    char buf[12];
    v = x;
    iptoa(buf, &v);
    return len(buf);
}

```

下面是`intlen`的部分代码,分别由带和不带栈保护者编译:


```

int intlen(long x)
x in %rdi
1  intlen:
2      subq    $40, %rsp
3      movq    %rdi, 24(%rsp)
4      leaq    24(%rsp), %rsi
5      movq    %rsp, %rdi
6      call    iptoa

```

a) 不带保护者

```

int intlen(long x)
x in %rdi
1  intlen:
2      subq    $56, %rsp
3      movq    %fs:40, %rax
4      movq    %rax, 40(%rsp)
5      xorl    %eax, %eax
6      movq    %rdi, 8(%rsp)
7      leaq    8(%rsp), %rsi
8      leaq    16(%rsp), %rdi
9      call    iptoa

```

b) 带保护者

- A. 对于两个版本：buf、v 和金丝雀值(如果有的话)分别在栈帧中的什么位置？
 B. 在有保护的代码中，对局部变量重新排列如何提供更好的安全性来对抗缓冲区越界攻击？

3. 限制可执行代码区域

最后一招是消除攻击者向系统中插入可执行代码的能力。一种方法是限制哪些内存区域能够存放可执行代码。在典型的程序中，只有保存编译器产生的代码的那部分内存才需要是可执行的。其他部分可以被限制为只允许读和写。正如第9章中会看到的，虚拟内存空间在逻辑上被分成了页(page)，典型的每页是2048或者4096个字节。硬件支持多种形式的内存保护，能够指明用户程序和操作系统内核所允许的访问形式。许多系统允许控制三种访问形式：读(从内存读数据)、写(存储数据到内存)和执行(将内存的内容看作机器级代码)。以前，x86体系结构将读和执行访问控制合并成一个1位的标志，这样任何被标记为可读的页也都是可执行的。栈必须是既可读又可写的，因而栈上的字节也都是可执行的。已经实现的很多机制，能够限制一些页是可读但是不可执行的，然而这些机制通常会带来严重的性能损失。

最近，AMD为它的64位处理器的内存保护引入了“NX”(No-Execute，不执行)位，将读和执行访问模式分开，Intel也跟进。有了这个特性，栈可以被标记为可读和可写，但是不可执行，而检查页是否可执行由硬件来完成，效率上没有损失。

有些类型的程序要求动态产生和执行代码的能力。例如，“即时(just-in-time)”编译技术为解释语言(例如Java)编写的程序动态地产生代码，以提高执行性能。是否能够将可执行代码限制在由编译器在创建原始程序时产生的那个部分中，取决于语言和操作系统。

我们讲到的这些技术——随机化、栈保护和限制哪部分内存可以存储可执行代码——是用于最小化程序缓冲区溢出攻击漏洞三种最常见的机制。它们都具有这样的属性，即不需要程序员做任何特殊的努力，带来的性能代价都非常小，甚至没有。单独每一种机制都降低了漏洞的等级，而组合起来，它们变得更加有效。不幸的是，仍然有方法能够攻击计算机[85, 97]，因而蠕虫和病毒继续危害着许多机器的完整性。

3.10.5 支持变长栈帧

到目前为止，我们已经检查了各种函数的机器级代码，但它们有一个共同点，即编译器能够预先确定需要为栈帧分配多少空间。但是有些函数，需要的局部存储是变长的。例如，当函数调用alloca时就会发生这种情况。alloca是一个标准库函数，可以在栈上分配任意字节数量的存储。当代码声明一个局部变长数组时，也会发生这种情况。

虽然本节介绍的内容实际上是如何实现过程的一部分，但我们还是把它推迟到现在才

讲，因为它需要理解数组和对齐。

图 3-43a 的代码给出了一个包含变长数组的例子。该函数声明了 n 个指针的局部数组 p ，这里 n 由第一个参数给出。这要求在栈上分配 $8n$ 个字节，这里 n 的值每次调用该函数时都会不同。因此编译器无法确定要给该函数的栈帧分配多少空间。此外，该程序还产生一个对局部变量 i 的地址引用，因此该变量必须存储在栈中。在执行工程中，程序必须能够访问局部变量 i 和数组 p 中的元素。返回时，该函数必须释放这个栈帧，并将栈指针设置为存储返回地址的位置。

```
long vframe(long n, long idx, long *q) {
    long i;
    long *p[n];
    p[0] = &i;
    for (i = 1; i < n; i++)
        p[i] = q;
    return *p[idx];
}
```

a) C代码

```
long vframe(long n, long idx, long *q)
n in %rdi, idx in %rsi, q in %rdx
Only portions of code shown
1  vframe:
2      pushq    %rbp                Save old %rbp
3      movq     %rsp, %rbp          Set frame pointer
4      subq     $16, %rsp            Allocate space for i (%rsp = s1)
5      leaq     22(,%rdi,8), %rax
6      andq     $-16, %rax
7      subq     %rax, %rsp            Allocate space for array p (%rsp = s2)
8      leaq     7(%rsp), %rax
9      shrq     $3, %rax
10     leaq     0(,%rax,8), %r8        Set %r8 to &p[0]
11     movq     %r8, %rcx             Set %rcx to &p[0] (%rcx = p)
    . . .
Code for initialization loop
i in %rax and on stack, n in %rdi, p in %rcx, q in %rdx
12     .L3:                          loop:
13         movq     %rdx, (%rcx,%rax,8)  Set p[i] to q
14         addq     $1, %rax              Increment i
15         movq     %rax, -8(%rbp)         Store on stack
16     .L2:
17         movq     -8(%rbp), %rax         Retrieve i from stack
18         cmpq     %rdi, %rax             Compare i:n
19         jl       .L3                   If <, goto loop
    . . .
Code for function exit
20     leave
21     ret                            Restore %rbp and %rsp
                                    Return
```

b) 生成的部分汇编代码

图 3-43 需要使用帧指针的函数。变长数组意味着在编译时无法确定栈帧的大小

为了管理变长栈帧，x86-64 代码使用寄存器 `%rbp` 作为帧指针 (frame pointer) (有时称为基指针 (base pointer))，这也是 `%rbp` 中 `bp` 两个字母的由来)。当使用帧指针时，栈帧的组织结构与图 3-44 中函数 `vframe` 的情况一样。可以看到代码必须把 `%rbp` 之前的值保存到栈中，因为它是一个被调用者保存寄存器。然后在函数的整个执行过程中，都使得 `%rbp` 指向那个时刻栈的位置，然后用固定长度的局部变量 (例如 `i`) 相对于 `%rbp` 的偏移量来引用它们。

图 3-43b 是 GCC 为函数 `vframe` 生成的部分代码。在函数的开始，代码建立栈帧，并为数组 `p` 分配空间。首先把 `%rbp` 的当前值压入栈中，将 `%rbp` 设置为指向当前的栈位置 (第 2~3 行)。然后，在栈上分配 16 个字节，其中前 8 个字节用于存储局部变量 `i`，而后 8 个字节是未被使用的。接着，为数组 `p` 分配空间 (第 5~11 行)。练习题 3.49 探讨了分配多少空间以及将 `p` 放在这段空间的什么位置。当程序到第 11 行的时候，已经 (1) 在栈上分配了 $8n$ 字节，并 (2) 在已分配的区域内放置好数组 `p`，至少有 $8n$ 字节可供其使用。

初始化循环的代码展示了如何引用局部变量 `i` 和 `p` 的例子。第 13 行表明数组元素 `p[i]` 被设置为 `q`。该指令用寄存器 `%rcx` 中的值作为 `p` 的起始地址。我们可以看到修改局部变量 `i` (第 15 行) 和读局部变量 (第 17 行) 的例子。`i` 的地址是引用 `-8(%rbp)`，也就是相对于帧指针偏移量为 `-8` 的地方。

在函数的结尾，`leave` 指令将帧指针恢复到它之前的值 (第 20 行)。这条指令不需要参数，等价于执行下面两条指令：

```
movq %rbp, %rsp    Set stack pointer to beginning of frame
popq %rbp           Restore saved %rbp and set stack ptr
                    to end of caller's frame
```

也就是，首先把栈指针设置为保存 `%rbp` 值的位置，然后把该值从栈中弹出到 `%rbp`。这个指令组合具有释放整个栈帧的效果。

在较早版本的 x86 代码中，每个函数调用都使用了帧指针。而现在，只在栈帧长可变的情况下才使用，就像函数 `vframe` 的情况一样。历史上，大多数编译器在生成 IA32 代码时会使用帧指针。最近的 GCC 版本放弃了这个惯例。可以看到把使用帧指针的代码和不使用帧指针的代码混在一起是可以的，只要所有的函数都把 `%rbp` 当做被调用者保存寄存器来处理即可。

 **练习题 3.49** 在这道题中，我们要探究图 3-43b 第 5~11 行代码背后的逻辑，它分配了变长大小的数组 `p`。正如代码的注释表明的， s_1 表示执行第 4 行的 `subq` 指令之后栈指针的地址。这条指令为局部变量 `i` 分配空间。 s_2 表示执行第 7 行的 `subq` 指令之后栈指针的值。这条指令为局部数组 `p` 分配存储。最后， p 表示第 10~11 行的指令赋给寄存器 `%r8` 和 `%rcx` 的值。这两个寄存器都用来引用数组 `p`。

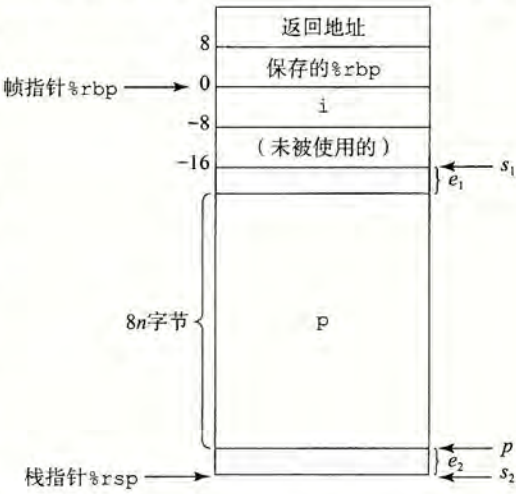


图 3-44 函数 `vframe` 的栈帧结构 (该函数使用寄存器 `%rbp` 作为帧指针。图右边的注释供练习题 3.49 所用)

图 3-44 的右边画出了 s_1 、 s_2 和 p 指示的位置。图中还画出了 s_2 和 p 的值之间可能有一个偏移量为 e_2 字节的位置，该空间是未被使用的。数组 p 的结尾和 s_1 指示的位置之间还可能有一个偏移量为 e_1 字节的地方。

- 用数学语言解释第 5~7 行中计算 s_2 的逻辑。提示：想想 -16 的位级表示以及它在第 6 行 `andq` 指令中的作用。
- 用数学语言解释第 8~10 行中计算 p 的逻辑。提示：可以参考 2.3.7 节中有关除以 2 的幂的讨论。
- 对于下面 n 和 s_1 的值，跟踪代码的执行，确定 s_2 、 p 、 e_1 和 e_2 的结果值。

n	s_1	s_2	p	e_1	e_2
5	2 065				
6	2 064				

- 这段代码为 s_2 和 p 的值提供了什么样的对齐属性？

3.11 浮点代码

处理器的浮点体系结构包括多个方面，会影响对浮点数据操作的程序如何被映射到机器上，包括：

- 如何存储和访问浮点数值。通常是通过某种寄存器方式来完成。
- 对浮点数据操作的指令。
- 向函数传递浮点数参数和从函数返回浮点数结果的规则。
- 函数调用过程中保存寄存器的规则——例如，一些寄存器被指定为调用者保存，而其他的被指定为被调用者保存。

简要回顾历史会对理解 x86-64 的浮点体系结构有所帮助。1997 年出现了 Pentium/MMX，Intel 和 AMD 都引入了持续数代的媒体(media)指令，支持图形和图像处理。这些指令本意是允许多个操作以并行模式执行，称为单指令多数据或 SIMD(读作 sim-dee)。在这种模式中，对多个不同的数据并行执行同一个操作。近年来，这些扩展有了长足的发展。名字经过了一系列大的修改，从 MMX 到 SSE(Streaming SIMD Extension，流式 SIMD 扩展)，以及最新的 AVX(Advanced Vector Extension，高级向量扩展)。每一代中，都有一些不同的版本。每个扩展都是管理寄存器组中的数据，这些寄存器组在 MMX 中称为“MM”寄存器，SSE 中称为“XMM”寄存器，而在 AVX 中称为“YMM”寄存器；MM 寄存器是 64 位的，XMM 是 128 位的，而 YMM 是 256 位的。所以，每个 YMM 寄存器可以存放 8 个 32 位值，或 4 个 64 位值，这些值可以是整数，也可以是浮点数。

2000 年 Pentium 4 中引入了 SSE2，媒体指令开始包括那些对标量浮点数据进行操作的指令，使用 XMM 或 YMM 寄存器的低 32 位或 64 位中的单个值。这个标量模式提供了一组寄存器和指令，它们更类似于其他处理器支持浮点数的方式。所有能够执行 x86-64 代码的处理器都支持 SSE2 或更高的版本，因此 x86-64 浮点数是基于 SSE 或 AVX 的，包括传递过程参数和返回值的规则[77]。

我们的讲述基于 AVX2，即 AVX 的第二个版本，它是在 2013 年 Core i7 Haswell 处理器中引入的。当给定命令行参数 `-mavx2` 时，GCC 会生成 AVX2 代码。基于不同版本的 SSE 以及第一个版本的 AVX 的代码从概念上来说类似的，不过指令名和格式有所不同。我们只介绍用 GCC 编译浮点程序时会出现的那些指令。其中大部分是标量 AVX 指令，我